

unit depends on, other conditions and dependencies, and gives a good description of the unit. That's it. It's about as complicated as a *.desktop* file.

Now think about it. These units describe dependencies. This means, units can be started in arbitrary order, as long as the dependencies are started before the unit in question. Also, two units are independent; they can be started in parallel. In fact, on my newly Systemd-fied system, as many as eight services start in parallel, near the middle of the process. No wonder 10 seconds have been shaved off, just like that!

Systemd does much more than just parallelise service start-up, though. It can even hold off from starting services until they're actually needed. Like Xinetd, units can be started on demand. Units can be mapped to UNIX file sockets as well as TCP sockets, and the first time the socket is accessed, the unit starts up. It's all about laziness—you hold up starting non-essential services for as long as possible.

Systemd also completely dispenses with the idea of runlevels; it implements targets, which are named, rather than numbered—and they, too, have dependencies. The graphical target depends on the multi-user target, which in turn depends on the network target, and so on. Bluetooth is a separate target. Basically, the idea is to have a specific capability (such as Bluetooth, networking, or X11) as a target, so you can switch targets on and off.

There's more. Systemd is highly integrated with both Udev and D-Bus. As a result, it brings back a lot of functionality that used to be the expertise of the deprecated HAL. Systemd handles filesystem mounts, and has a built-in auto-mounter. It handles the *fstab* file itself, without depending on *mount*. Systemd also takes over the work of ConsoleKit, so GNOME 3.4 (which can work with this functionality now) doesn't need ConsoleKit to handle logins and seats. In fact, ConsoleKit isn't even actively maintained any more.

And now, let's look at the final trick that Systemd has up its sleeve. A lot of services use double-forking to daemonise themselves. It's very hard to kill these services when they misbehave, for the simple reason that after a double-fork, these processes kind of break out of the process hierarchy, and the *init* system doesn't know where the process is. Systemd solves this problem by using CGroups and letting the kernel keep track of the processes. Now this might alarm users of BFS, which is famous for 'not implementing CGroups' (I use BFS too), but BFS does implement CGroups—it just doesn't implement fancy scheduling based on CGroups.

Getting Systemd

If you're using Fedora, you already have Systemd, in which case you can head on to the next section. If you're using any of the other major distros, you're out of luck—you can't currently use Systemd, except if you're using OpenSUSE, in which case the next version of OpenSUSE will use Systemd, so you could upgrade to Factory.

Of course, using Arch Linux has its advantages. I'm free

to choose every single component of my system... including my *init* system. I had the death wish to completely replace SysV-init with Systemd—and I almost came to regret it. But the end result was entirely worth it.

Now, you've got to realise that Systemd doesn't use any of the old SysV-init files. It doesn't use *inittab*, it doesn't use *rc.conf*, and it doesn't use *rc.local*. You've got to offload *rc.conf* and *rc.local* into other files before you can start using Systemd. In short, before you can actually install Systemd, you have to prepare the system. Start by creating */etc/vconsole.conf*:

```
KEYMAP=us
FONT=lat9w-16
FONT_MAP=8859-1_to_uni
```

Once that's done, set a hostname in */etc/hostname*:

```
my-awesome-archbox
```

Now, set up locale settings in */etc/locale.conf*:

```
LANG=en_IN.utf8
LC_CTYPE="en_IN.utf8"
LC_NUMERIC="en_IN.utf8"
LC_TIME="en_IN.utf8"
LC_COLLATE="C"
LC_MONETARY="en_IN.utf8"
LC_MESSAGES="en_IN.utf8"
LC_PAPER="en_IN.utf8"
LC_NAME="en_IN.utf8"
LC_ADDRESS="en_IN.utf8"
LC_TELEPHONE="en_IN.utf8"
LC_MEASUREMENT="en_IN.utf8"
LC_IDENTIFICATION="en_IN.utf8"
LC_ALL=
```

And add this line to the top of */etc/environment*:

```
LANG=en_IN.utf8
```

Though *en_IN* may not be available by default, you can enable it by uncommenting the relevant lines in */etc/locale.gen* and running *sudo locale-gen*. The *locale.gen* file is self-explanatory.

Systemd uses files in the */etc/modules-load.d/* directory to load up extra kernel modules. I use FUSE, which isn't loaded by default, so I have a file at */etc/modules-load.d/fuse.conf*, with the following line:

```
fuse
```

We'll need to integrate the Systemd journal into Syslog-NG, so open up */etc/syslog-ng/syslog-ng.conf*, find the *src* section, and replace it all with: